endava

Agenda

01 JSX and Functional components

02 React Hooks(useReducer, useMemo, useCallback)

03 Context API and composition patterns

04 Custom hook design and reuse

05 Rendering patterns(Controlled and uncontrolled, HOC’s)

¿Qué es JSX exactamente?

Hace que React más intuitivo

01 Extensión de JavaScript

JSX extiende JavaScript permitiendo escribir componentes de interfaz de usuario de forma clara y legible, como si fuera HTML pero con toda la potencia de JavaScript.

02 Describe la estructura UI

Permite describir la estructura de la interfaz de manera intuitiva y visual, facilitando la comprensión de cómo se verá el componente final.

03 Transpilación a JavaScript

El navegador no entiende JSX directamente. Se traduce a JavaScript puro mediante herramientas como Babel para su ejecución en el navegador.

Del JSX al navegador: Transpilación

1

Código JSX escrito

El desarrollador escribe componentes usando la sintaxis JSX, que combina JavaScript con etiquetas similares a HTML para mayor claridad.

2

Babel transpila

Babel actúa como transpilador, convirtiendo cada etiqueta JSX en llamadas a React.createElement() que el navegador puede ejecutar.

3

JavaScript puro ejecutado

El navegador recibe JavaScript estándar con funciones React.createElement() que reciben tipo de elemento, props e hijos como argumentos.

Babel

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Functional components

```
import { useState } from "react";

function ContadorFuncional() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Contador Funcional: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Incrementar
      </button>
    </div>
  );
}

export default ContadorFuncional;
```

⭐ 1. Para simplificar el código

Los componentes de clase eran verbosos:

- `constructor`
- `this.state`
- `this.setState`
- binding de métodos (`this.handleClick = this.handleClick.bind(this)`)

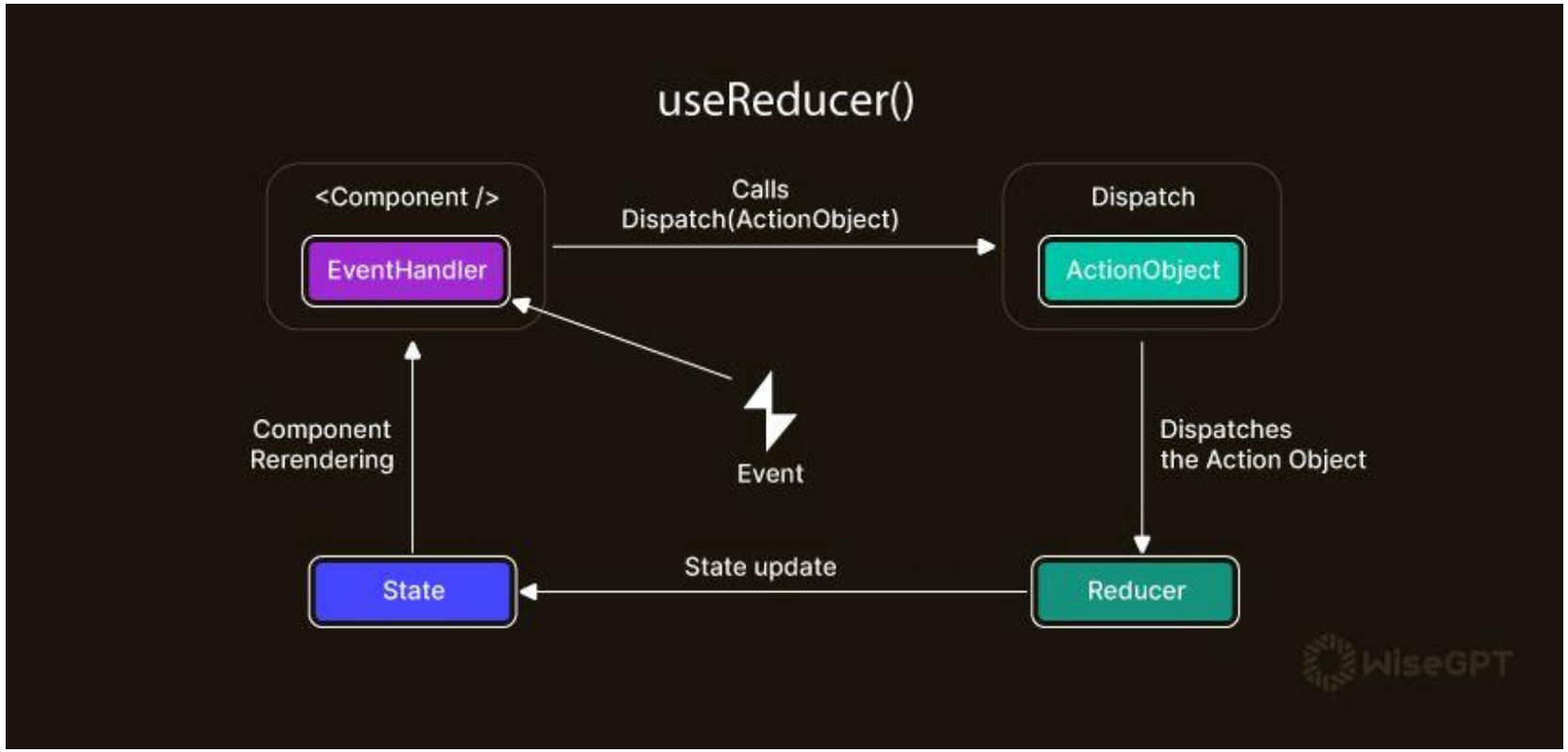
Los Functional Components eliminan todo eso.
Menos código, menos errores, más claridad.

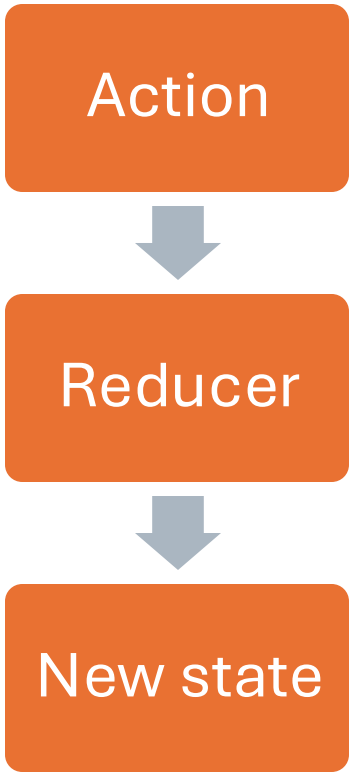
```
import React, { Component } from "react";

class ContadorClase extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  incrementar = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <h2>Contador Clase: {this.state.count}</h2>
        <button onClick={this.incrementar}>
          Incrementar
        </button>
      </div>
    );
  }
}

export default ContadorClase;
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Advanced hooks: useReducer

Hook de React para gestionar estados complejos o lógica de actualización estructurada, funcionando como alternativa a useState





© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Advanced hooks: useReducer

Caso	Usar
Estado Simple	useState
Muchos cambios de estado	useReducer
Logica compleja	useReducer
Estado compartido con Context	useReducer

useReducer no es para reemplazar useState. Es para cuando la lógica del estado empieza a crecer y necesitamos una forma más predecible y organizada de manejarla

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

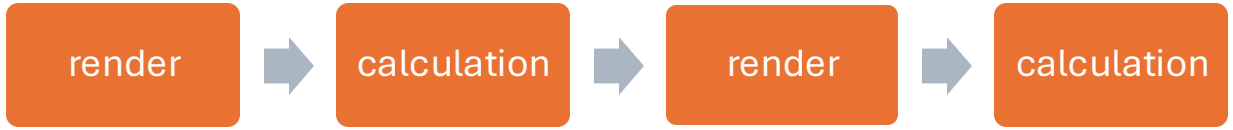
Advanced hooks: useMemo

Hook de React que optimiza el rendimiento al memorizar (cachear) el resultado de un cálculo costoso entre renderizados

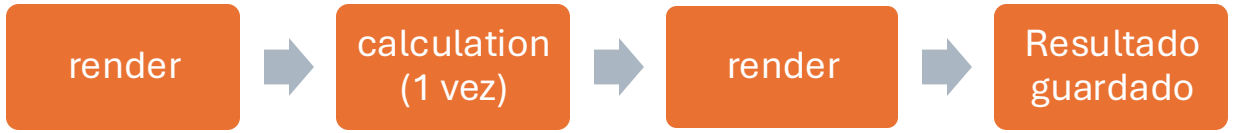
main.js

```
1  function App() {
2    const numbers = [1,2,3,4,5]
3
4    const total = numbers.reduce((a,b) => a + b)
5
6    return <div>Total: {total}</div>
7  }
8
9
```

Sin useMemo



Con useMemo



© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Advanced hooks: useMemo

Hook de React que optimiza el rendimiento al memorizar (cachear) el resultado de un cálculo costoso entre renderizados

Caso	Usar useMemo
Calculos costosos	☑
Filtrar listas grandes	☑
Transformar datos pesados	☑
Valores simples	⊘

useMemo ayuda a evitar cálculos innecesarios cuando un componente renderiza muchas veces.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Advanced hooks: useCallback

Si useMemo memoriza **valores**, useCallback memoriza **funciones**.

En React, **cada render crea nuevas funciones**.

```javascript
function App() {
  const handleClick = () => {
    console.log("clicked");
  };

  return <button onClick={handleClick}>Click</button>;
}
```

Cada render crea:
handleClick (nueva referencia en memoria)

Normalmente **esto no es un problema**, pero puede afectar cuando:

- Pasas funciones a **componentes hijos**
- Esos hijos usan **React.memo**

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

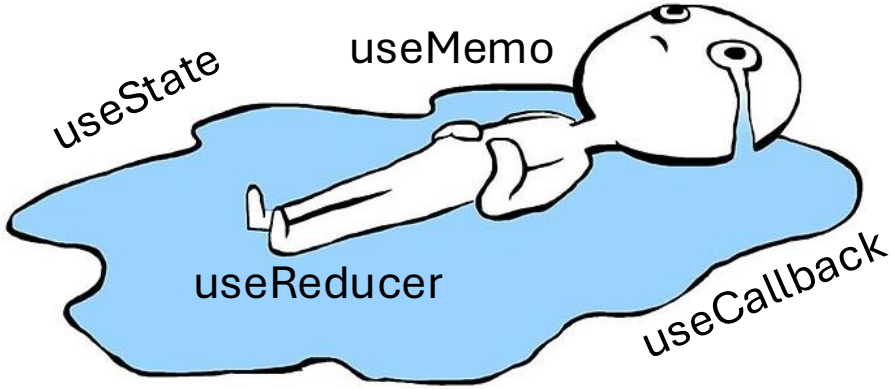
Advanced hooks: useCallback

Caso	Hook
Pasar funciones a hijos	☑
Componentes con React.memo	☑
Callbacks en listas grandes	☑
Funciones simples internas	⊘

Si no tienes un problema de renderización, probablemente no necesitas useCallback.
useCallback evita que una función cambie de referencia entre renders.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Hasta ahora



Hook	Uso
useState	manejar estado
useReducer	lógica compleja
useMemo	memorizar valores
useCallback	memorizar funciones

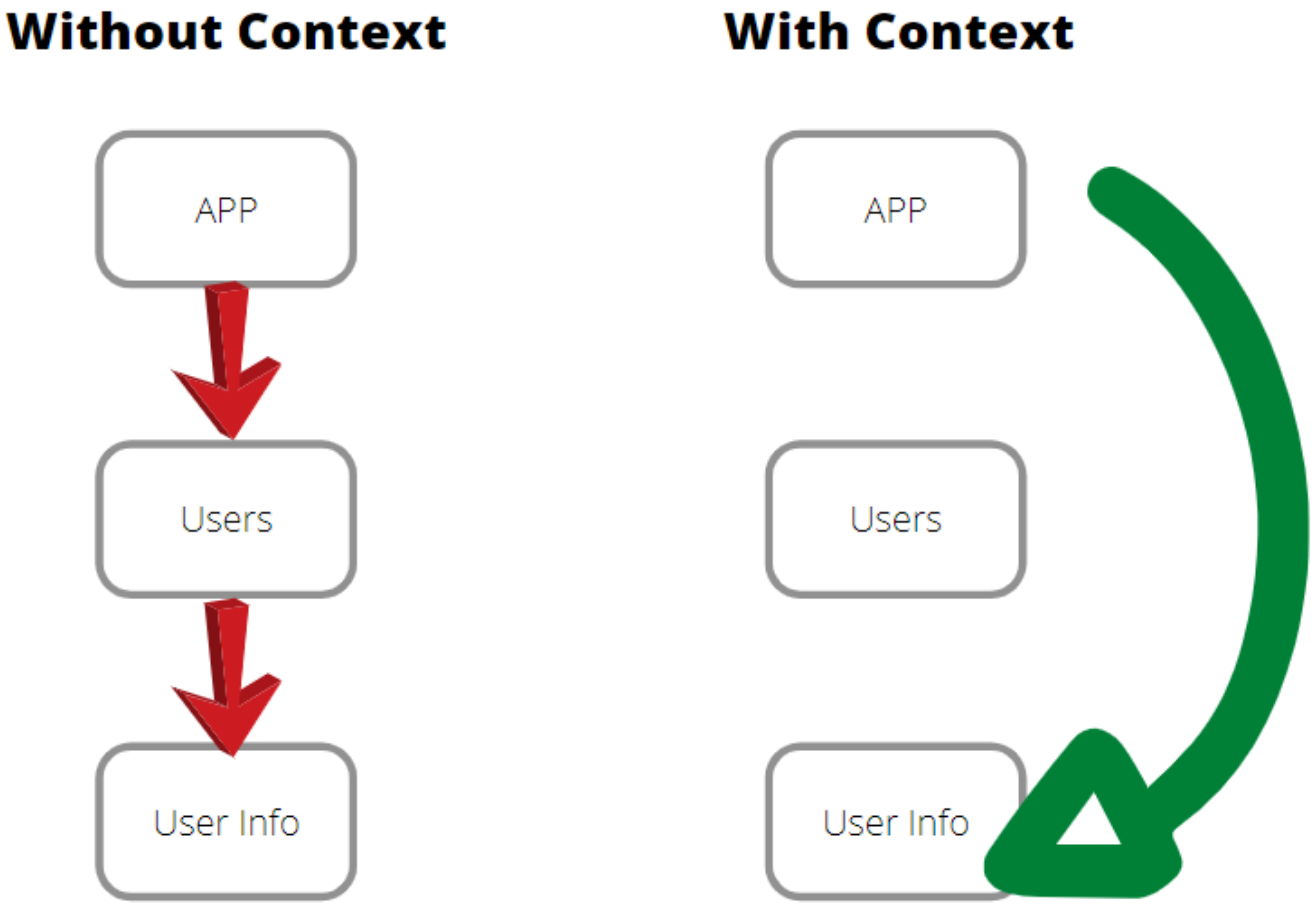
© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Context API

Context permite **compartir datos globalmente sin pasar props manualmente**.

Prop Drilling

```
<App user={user}>
  <Dashboard user={user}>
    <Sidebar user={user}>
      <Profile user={user} />
    </Sidebar>
  </Dashboard>
</App>
```



© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Context API

Caso	Usar
tema (dark/light)	☑
usuario autenticado	☑
idioma	☑
estado global complejo	mejor Zustand/Redux

Redux

Context API funciona como **WiFi**. Una vez conectado, todos los dispositivos pueden usar la red sin cables.

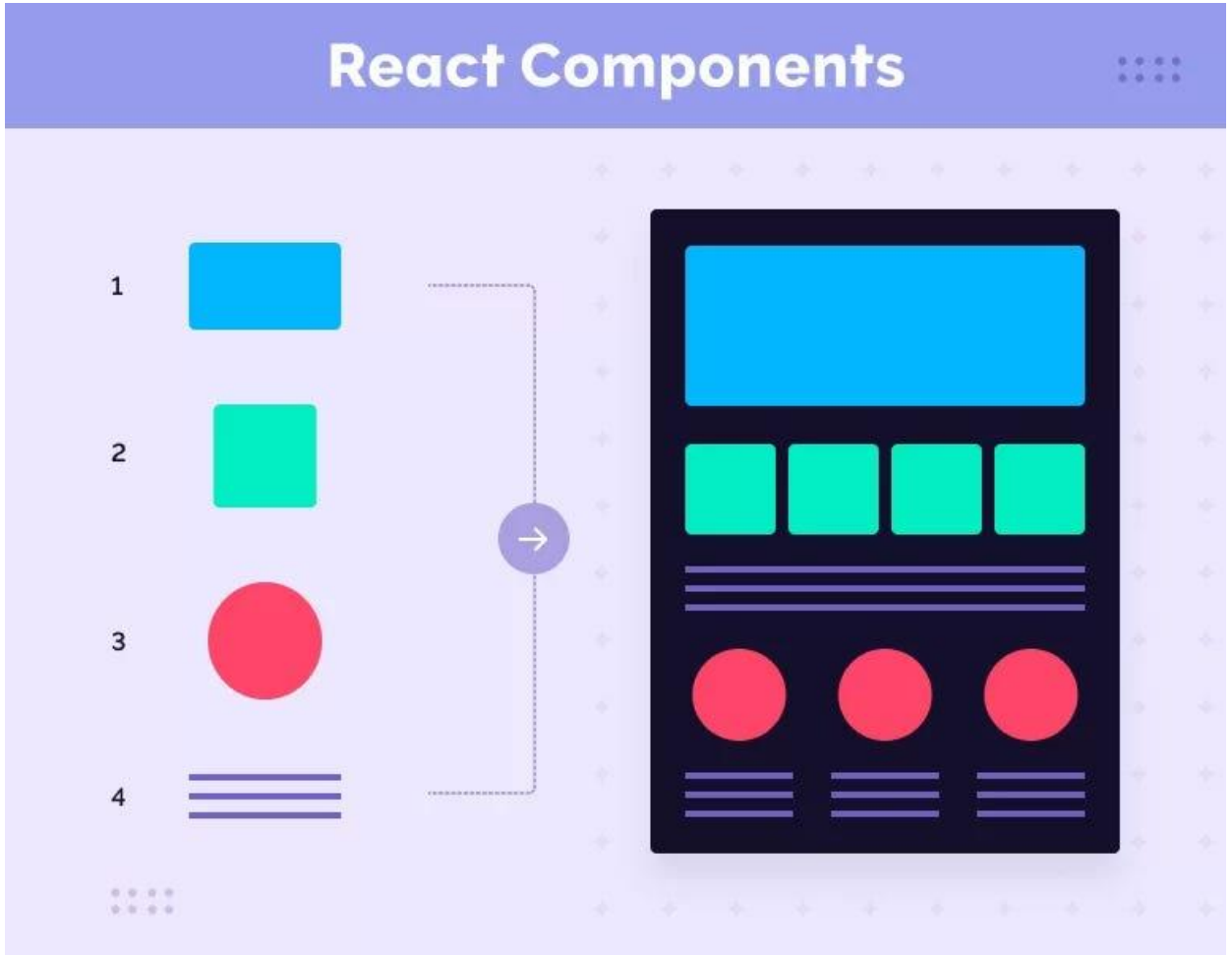
© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Compositions Patterns

React no promueve herencia, promueve **composition**.

Que significa esto:

Construir componentes grandes
usando componentes pequeños



© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Compositions Patterns

Ejemplo básico de composición

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}

export default function App() {
  return (
    <>
      <Card>
        <h2>Title</h2>
        <p>Description</p>
      </Card>
    </>
  )
}
```

Aquí **Card no sabe qué contiene**.

Children Pattern

```
function Layout({ children }) {
  return (
    <div>
      <Header />
      {children}
      <Footer />
    </div>
  );
}

export default function App() {
  return (
    <Layout>
      <Dashboard />
    </Layout>
  )
}
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Compositions Patterns

Compound Components

```
1  export default function App() {
2    return (
3      <Tabs>
4        <Tabs.List>
5          <Tabs.Tab>Home</Tabs.Tab>
6          <Tabs.Tab>Profile</Tabs.Tab>
7        </Tabs.List>
8
9        <Tabs.Panels>
10         <Tabs.Panel>Home content</Tabs.Panel>
11         <Tabs.Panel>Profile content</Tabs.Panel>
12       </Tabs.Panels>
13     </Tabs>
14   )
15 }
16
```

Ejemplo de **Compound Component Pattern** + **Context**, algo muy común en librerías de UI

Radix

HeadlessUI

Material UI

Beneficios:

componentes **reutilizables**
código **más flexible**
arquitectura **más limpia**

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Custom hooks

Antes de los custom hooks, cuando varios componentes necesitaban **la misma lógica**, se duplicaba código.

```
function ComponentA() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);

    return () => window.removeEventListener("resize", handleResize);
  }, []);
}

function ComponentB() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);

    return () => window.removeEventListener("resize", handleResize);
  }, []);
}
```

Un Custom Hook es simplemente una función que usa hooks de React.

```
import { useState, useEffect } from "react";

function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return width;
}

function ComponentA() {
  const width = useWindowWidth();

  return <h1>Width: {width}</h1>;
}
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Rendering patterns Controlled and uncontrolled

Cuando trabajamos con formularios en React, hay dos formas de manejar los inputs: **controlled** y **uncontrolled**.

Tipo	Quién controla el estado
Controlled	React
Uncontrolled	El DOM

```
1  import { useState } from "react";
2
3  function ControlledInput() {
4    const [name, setName] = useState("");
5
6    return (
7      <input
8        value={name}
9        onChange={(e) => setName(e.target.value)}
10     />
11   );
12 }
13
14 export default ControlledInput
```

React siempre sabe **qué valor tiene el input**.

```
1  import { useRef } from "react";
2
3  function UncontrolledInput() {
4    const inputRef = useRef();
5
6    function handleSubmit() {
7      console.log(inputRef.current.value);
8    }
9
10   return (
11     <>
12       <input ref={inputRef} />
13       <button onClick={handleSubmit}>Submit</button>
14     </>
15   );
16 }
```

El DOM funciona de forma independiente, React solo consulta el valor cuando lo necesita.

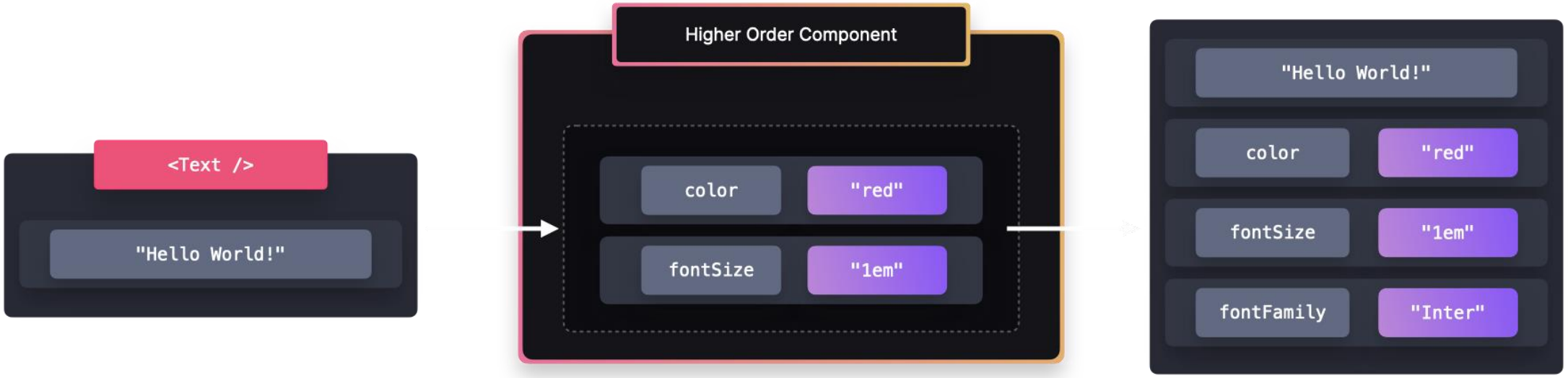
React Hook Form

FORMIK

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

HOC pattern

Un Higher-Order Component es una función que recibe un componente y retorna un nuevo componente con funcionalidades adicionales.



© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

HOC pattern

Problema de los HOCs:

Cuando se usan muchos, el código se vuelve difícil de leer.

```
1  export default withRouter(
2    withAuth(
3      withTheme(
4        Dashboard
5      )
6    )
7  );
8
```

Evolución de patrones en React:

```
Mixins (antiguo)
↓
HOC
↓
Render Props
↓
Custom Hooks (actual)
```

HOCs fueron una solución poderosa para reutilizar lógica antes de los hooks, pero hoy en día la mayoría de los casos se resuelven mejor con Custom Hooks.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0